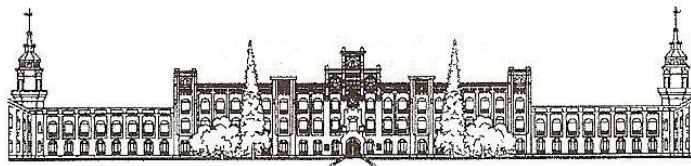


Міністерство освіти і науки України
Національний технічний університет
України
«Київський політехнічний інститут імені Ігоря Сікорського»



Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних
систем

ДКР

з дисципліни

«Моделювання та оптимізація комп'ютерних систем»

Виконала студентка групи:

КВ-11 Михайліченко Софія

Перевірив: Малезик П.М.

Оцінка:

Київ – 2024

Варіант 14:

Завдання:

Проектування оптимальних цифрових фільтрів Вінера в системі MATLAB

Теоретичні відомості:

Введення в цифрові фільтри:

У сучасному світі, де інформація представлена у вигляді сигналів, фільтри відіграють важливу роль в її обробці та аналізі. Їхні можливості дозволяють видобувати корисну інформацію з шуму, покращувати якість переданих даних та створювати нові сигнали з заданими характеристиками.

По суті, фільтр – це система або мережа, яка вибірково змінює форму сигналу (амплітудно-частотну або фазово-частотну характеристику). Основними цілями фільтрації є покращення якості сигналу (наприклад, усунення або зниження перешкод), вилучення з сигналів інформації або поділ кількох сигналів, об'єднаних раніше для, наприклад, ефективного використання доступного каналу зв'язку.

Фільтри можна використовувати для різних цілей, зокрема:

- Покращення якості сигналу: видалення шуму, перешкод або інших небажаних компонентів з сигналу.
- Вилучення інформації: виділення певних частотних компонентів з сигналу, які несуть корисну інформацію.
- Розділення сигналів: роз'єднання декількох сигналів, які були раніше об'єднані.
- Формування сигналу: створення нових сигналів з певними характеристиками.

Фільтри відіграють важливу роль у системах обробки сигналів, і вони бувають різних типів залежно від призначення. Низькочастотні фільтри (LPF) пропускають сигнали з частотами нижче певної порогової частоти, одночасно пригнічуючи сигнали з вищими частотами. *Високочастотні фільтри (HPF)*,

навпаки, пропускають сигнали з частотами вище певної порогової частоти, пригнічуючи нижчі частоти. *Смугові фільтри (BPF)* пропускають сигнали в певному діапазоні частот, блокуючи сигнали за межами цього діапазону. *Загороджувальні фільтри (BSF)* або фільтри відсічення, навпаки, блокують сигнали в певному діапазоні частот, пропускаючи всі інші.

Комбінація фільтрів різних типів дозволяє створювати складні системи фільтрації для специфічних застосувань. *Адаптивні фільтри* змінюють свої характеристики в залежності від вхідного сигналу, що дозволяє їм ефективно працювати в умовах змінних характеристик сигналу та шуму. *Фірмові фільтри (FIR)* та *рекурсивні фільтри (IIR)* представляють дві основні категорії цифрових фільтрів за методами реалізації. FIR-фільтри мають скінченну імпульсну характеристику і завжди стабільні, тоді як IIR-фільтри мають нескінченну імпульсну характеристику, що дозволяє їм бути більш ефективними з точки зору використання обчислювальних ресурсів. *Калманівський фільтр* є прикладом складного адаптивного фільтра, що використовується для оцінки стану системи в умовах шуму та невизначеності. *Фільтр Вінера* оптимальний для зменшення шуму в сигналах при наявності стаціонарного шуму та корисного сигналу.

Цифровий фільтр – це математичний алгоритм, реалізований на апаратному та/або програмному рівні, який із заданою метою діє на вхідний і генерує вихідний цифровий сигнал. Терміном “цифровий фільтр” позначають певну апаратну або програмну процедуру, реалізацію алгоритму фільтрації. У цифрових фільтрах часто використовуються оцифровані аналогові сигнали або числа, які просто зберігаються в пам'яті комп'ютера і представляють деякі змінні.

Важливу роль цифровій обробці сигналів грають цифрові фільтри. У порівнянні з аналоговими фільтрами вони переважні в безлічі областей (наприклад, стиснення даних, біомедична обробка сигналів, обробка мови,

обробка зображень, передача даних, цифрове аудіо, телефонне ехопридушення), так як мають ряд переваг.

У нашому завданні потрібно знати, що таке оптимальний фільтр – це стаціонарна лінійна частотно-виборча система, яка виконує обробку зашумленого сигналу найкращим способом, тобто $(с/ш) = \max$.

Оптимальні фільтри проектуються на основі статистичних характеристик сигналу та шуму. Їх характеристики, такі як частотна характеристика та імпульсна характеристика, обираються таким чином, щоб максимально покращити співвідношення сигнал/шум на виході фільтра.

Розглянемо детальніше фільтр Вінера:

Завдання фільтра Вінера полягає в отриманні передавальної функції фільтра, що забезпечує найкращу за критерієм мінімуму середньоквадратичної помилки фільтрацію корисного сигналу, при впливі на нього адитивного випадкового шуму. Адаптивні фільтри, можна розглядати як наближену, більш просту для практики реалізацію лінійного оптимального фільтра Вінера. Завдання вперше була вирішена незалежно двома вченими:

- Радянським математиком А. Колмогоровим, який опублікував рішення в 1941 року в статті «Інтерполяція і екстраполяція стаціонарних випадкових послідовностей»
- Американським вченим - математиком Н. Вінером, що опублікували результат в 1949 г. в статті «The Extrapolation, Interpolation, and Smoothing of Stationary Time Series with Engineering Applications»).

Тому відповідні оптимальні фільтри отримали назву фільтрів Вінера - Колмогорова. І таку назву зустрічається у багатьох публікаціях. Але частіше використовується назва «фільтри Вінера». Мабуть, причинами такої термінології є та обставина, що стаття А. Колмогорова - це теоретична робота вченого - математика. Для інженерів - практиків вона виявилася мало доступною. Крім того, російська мова менш поширена в порівнянні з англійською. Тому більш широку популярність і розуміння знайшли результати роботи Н. Вінера, хоча вона була опублікована пізніше

Типові детерміновані фільтри розроблені для отримання бажаної частотної характеристики. Однак алгоритм фільтра Вінера використовує інший підхід. Передбачається, що ми маємо початкові дані про спектральні властивості вихідного сигналу та шуму, а інший підхід шукає лінійний інваріантний до часу фільтр, вихід якого буде максимально наближений до вихідного сигналу.

Фільтр Вінера характеризується наступним:

1. Припущення: сигнал та (адитивний) шум - це стаціонарні лінійні випадкові процеси з відомими спектральними характеристиками або з відомою автокореляцією та взаємкореляцією.
2. Вимога: фільтр повинен бути фізично реалізованим.
3. Критерій ефективності: мінімальна середньоквадратична помилка (MMSE). Цей фільтр часто використовується в процесі деконволюції.

Оптимальний фільтр Вінера:

Перш ніж власне розглядати алгоритми адаптації, необхідно встановити оптимальні параметри фільтра, до яких ці алгоритми повинні наближуватись. Підхід до задачі оптимальної фільтрації може бути як статистичним, так і детермінованим. Спочатку розглянемо статистичний варіант.

Якщо випадковий дискретний сигнал $x(k)$ оброблюється нерекурсивним дискретним фільтром N -го порядку, коефіцієнти якого можуть бути представлені векторним стовбцем $w = [w_0, w_1, \dots, w_N]^T$. Вихідний сигнал фільтра дорівнює:

$$y(k) = \mathbf{u}^T(k)w$$

$\mathbf{u}(k) = [x(k), x(k-1), \dots, x(k-N)]^T$ – де вектор стовбець вмісті лінії затримки фільтра на k -му кроці.

Передусім, є сигнал $d(k)$ який являється також випадковим. Помилка результату такого сигналу дорівнює:

$$e(k) = d(k) - y(k) = d(k) - \mathbf{u}^T(k)w$$

Потрібно визначити коефіцієнти фільтру w , які будуть забезпечувати максимальну ідентичність вихідного сигналу до зразкового фільтру, тобто які максимально зменшать похибку $e(k)$. Оскільки $e(k)$ також є випадковим

процесом, в якості запобіжної величини розумно прийняти середній квадрат.

Таким чином, оптимізований функціонал виглядає так:

$$J(\mathbf{w}) = e^2(k) \rightarrow \min$$

Квадрат похибки дорівнює:

$$e^2(k) = (d(k) - \mathbf{u}^T(k)\mathbf{w})^2 = d^2(k) - 2d(k)\mathbf{u}^T(k)\mathbf{w} + \mathbf{w}^T\mathbf{u}(k)\mathbf{u}^T(k)\mathbf{w}.$$

Статистично усереднити даний вираз, отримаємо:

$$J(\mathbf{w}) = \overline{e^2(k)} = \overline{d^2(k)} - \overline{2d(k)\mathbf{u}^T(k)\mathbf{w}} + \mathbf{w}^T\overline{\mathbf{u}(k)\mathbf{u}^T(k)}\mathbf{w}.$$

Вхідні величини в отриману формулу несуть такий сенс:

- $\overline{d^2(k)} = \sigma_d^2$ – середній квадрат зразкового сигналу;
- $\overline{d(k)\mathbf{u}^T(k)} = \mathbf{p}^T$ – транспонований векторстовбець $\mathbf{p}$ взаємо-кореляцій між k -м відліком зразкового сигналу та вмістом лінії затримки фільтру. Якщо розглянуті випадкові процеси $x(t)$ та $d(k)$ є спільно стаціонарними, то вектор взаємних кореляцій не залежить від номеру кроку k ;
- $\overline{\mathbf{u}(k)\mathbf{u}^T(k)} = \mathbf{R}$ – кореляційна матриця з розміром $(N+1) \times (N+1)$.

$$\mathbf{R} = \begin{bmatrix} R_x(0) & R_x(1) & R_x(2) & \dots & R_x(N) \\ R_x(1) & R_x(0) & R_x(1) & \dots & R_x(N-1) \\ R_x(2) & R_x(1) & R_x(0) & \dots & R_x(N-2) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ R_x(N) & R_x(N-1) & R_x(N-2) & \dots & R_x(0) \end{bmatrix},$$

тут $R_x(m) = \overline{x(k)x(k-m)}$ — кореляційна функція (КФ) випадкового процесу $\{x(k)\}$.

З урахуванням введених позначень рівняння приймає наступний вид:

$$J(\mathbf{w}) = \sigma_d^2 - 2\mathbf{p}^T\mathbf{w} + \mathbf{w}^T\mathbf{R}\mathbf{w}.$$

Дане рівняння представляє форму щодо $\mathbf{w}$, тому матриця $\mathbf{R}$ має єдиний мінімум. Щоб знайти розв'язок цього рівняння потрібно прирівняти цей єдиний мінімум до нуля вектор градієнту:

$$\mathbf{grad} J(\mathbf{w}) = -2\mathbf{p} + 2\mathbf{R}\mathbf{w} = 0.$$

Таке рівняння називають рівнянням Вінера-Хопфа. Задача адаптивної фільтрації полягає в налаштуванні вагових коефіцієнтів $w(0), w(1) \dots w(N-1)$ за допомогою необхідного алгоритму для досягнення оптимальної точки на поверхні продуктивності фільтра Вінера.

Звідси отримаємо рішення для оптимальних коефіцієнтів фільтру:

$$\mathbf{w} = \mathbf{R}^{-1} \mathbf{p}.$$

Такий фільтр називається фільтром Вінера. Підстановка рівняння дає мінімально досягну дисперсію сигналу похибки.

На фільтр одночасно подається два сигнали y_k та x_k . Зазвичай y_k складається з компонента, який корелює із x_k і компонента, який з x_k не корелює. Фільтр Вінера видає оптимальну оцінку тої частоти y_k , який корелює з x_k , потім ця оцінка віднімається із y_k і отримуємо e_k .

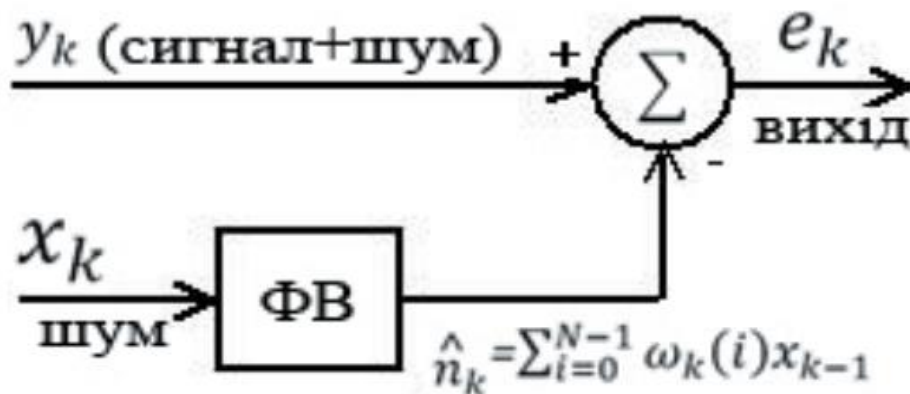


Рис. 1. Фільтр Вінера

Практичне використання фільтра Вінера є обмеженим із-за наявності таких недоліків.

- необхідно використовувати автокореляційну матрицю $\mathbf{R}$ взаємної кореляції ρ , але ці обидві величини треба визначати;
- фільтри вимагають транспонування матриць, що є трудомістким процесом навіть для ЕОМ.
- якщо сигнали не стаціонарні, то $\mathbf{R}$ і ρ є змінними функціями, а це вимагає багаторазового визначення $\mathbf{w}_{\text{опт}}$.

Перш ніж приступити до проектування фільтрів розберемо етапи їх розробки, які нам потрібно знати та використати в практичній частині.

Розробка цифрового фільтра відбувається у п'ять етапів.

1. Специфікація вимог до фільтру.
2. Обчислення відповідних коефіцієнтів фільтра.
3. Подання фільтра відповідною структурою.
4. Аналіз впливу кінцевої розрядності на продуктивність фільтра.
5. Реалізація фільтра на програмному та/або апаратному рівні.

Названі п'ять етапів який завжди незалежні; крім того, вони не завжди розташовуються у вказаному порядку. Фактично існують методи, які дозволяють об'єднати другий етап та деякі аспекти третього та четвертого. Підхід, викладений у даній лекції, - це просте покрокове керівництво, що гарантує успішну структуру. Щоб отримати ефективний фільтр, іноді доводиться проводити цей процес у кілька ітерацій, особливо, якщо специфікації не є цілком певними (як зазвичай і буває), або розробник бажає досліджувати альтернативні структури. Детальний опис названих п'яти етапів наводиться нижче.

Огляд основних функцій та інструментів для обробки сигналів MATLAB

Для аналізу використовуватимемо пакет ***Signal Analyzer App***

Типовий робочий процес для інспектування та порівняння сигналів за допомогою Signal Analyzer App включає:

1. **Вибір сигналів для аналізу** — можна вибрати будь-який сигнал, доступний у робочій області MATLAB. Додаток приймає числові масиви та сигнали з вбудованою часовою інформацією, такі як MATLAB timetable масиви, об'єкти timeseries та об'єкти labeledSignalSet. Для отримання додаткової інформації див. Data Types Supported by Signal Analyzer. Додавання часової інформації до сигналів за допомогою частоти дискретизації, числових векторів, масивів тривалості або виразів MATLAB.

2. **Передобробка сигналів** — редагування сигналів за допомогою обрізки, розрізання, вилучення або обрізання дій. Фільтрування сигналів за допомогою низькочастотних, високочастотних, смугових або режекторних фільтрів. Видалення трендів та обчислювання оболонки сигналів. Згладжування сигналів за допомогою рухомих середніх, регресії, фільтрів Савітцкі-Голая або інших методів. Видалення шумів за допомогою вейвлетів. Знаходження та заповнювання відсутніх даних за допомогою рухомого середнього, авторегресійної моделі або інших методів (з R2024a). Змінювання частоти дискретизації сигналів або інтерполювання нерівномірно дискретизованих сигналів на рівномірні сітки. Передоброблення сигналів за допомогою власних користувацьких функцій. Генеруйте функції MATLAB для автоматизації операцій передобробки.
3. **Дослідження сигналів** — Відтворювання аудіосигналів (з R2024a). Створення графіків та порівняння даних, їхні спектри, спектрограми або скалограми. Пошук особливості та закономірності у часовій області, частотній області та часо-частотній області. Контроль витоку, ширини смуги роздільної здатності або довжини вікна при обчисленні спектрів потужності. Обчислення спектрів наполегливості для аналізу спорадичних сигналів та уточнюйте оцінки спектрограм за допомогою повторного призначення. Вилучайте області інтересу з сигналів.
4. **Вимірювання сигналів** — Інтерактивне вимірювання даних сигналів, спектрів та часо-частотних даних. Обчислення статистики сигналів у часовій області, таку як мінімум, максимум, середнє та середньоквадратичне значення. Знаходження та анотування піки сигналу в часовій області. Перегляд статистики та значення піків у таблиці вимірювань.
5. **Обмін аналізом** — Копіювання дисплеїв з додатку до буфера обміну як зображення. Експорт сигналів до робочої області MATLAB або зберігання їх у MAT-файлах. Генерування скриптів MATLAB для

автоматизації обчислення спектру потужності, спектрограми або оцінок спектру наполегливості та вилучення областей інтересу. Зберігання сесії Signal Analyzer, щоб відновити свій аналіз пізніше або на іншому комп'ютері.

Обробка сигналів в MATLAB включає використання різноманітних вбудованих функцій для аналізу, фільтрації та модифікації сигналів. Ось детальний опис деяких основних функцій, таких як `filter`, `xcorr`.

Розберемо основні функції обробки сигналів, які потрібні для написання моделі фільтра Вінера в MATLAB:

Обробка сигналів в MATLAB включає використання різноманітних вбудованих функцій для аналізу, фільтрації та модифікації сигналів. Ось детальний опис деяких основних функцій, таких як `filter`, `xcorr`.

Функція `filter` використовується для фільтрації сигналів за допомогою різних фільтрів (KIX, БІХ). Вона застосовує фільтр до вхідного сигналу на основі передаточної функції, визначеної коефіцієнтами чисельника і знаменника.

Синтаксис:

$y = \text{filter}(b, a, x)$

- b - Коефіцієнти чисельника передаточної функції фільтра.
- a - Коефіцієнти знаменника передаточної функції фільтра.
- x - Вхідний сигнал.
- y - Вихідний (фільтрований) сигнал.

Приклад:

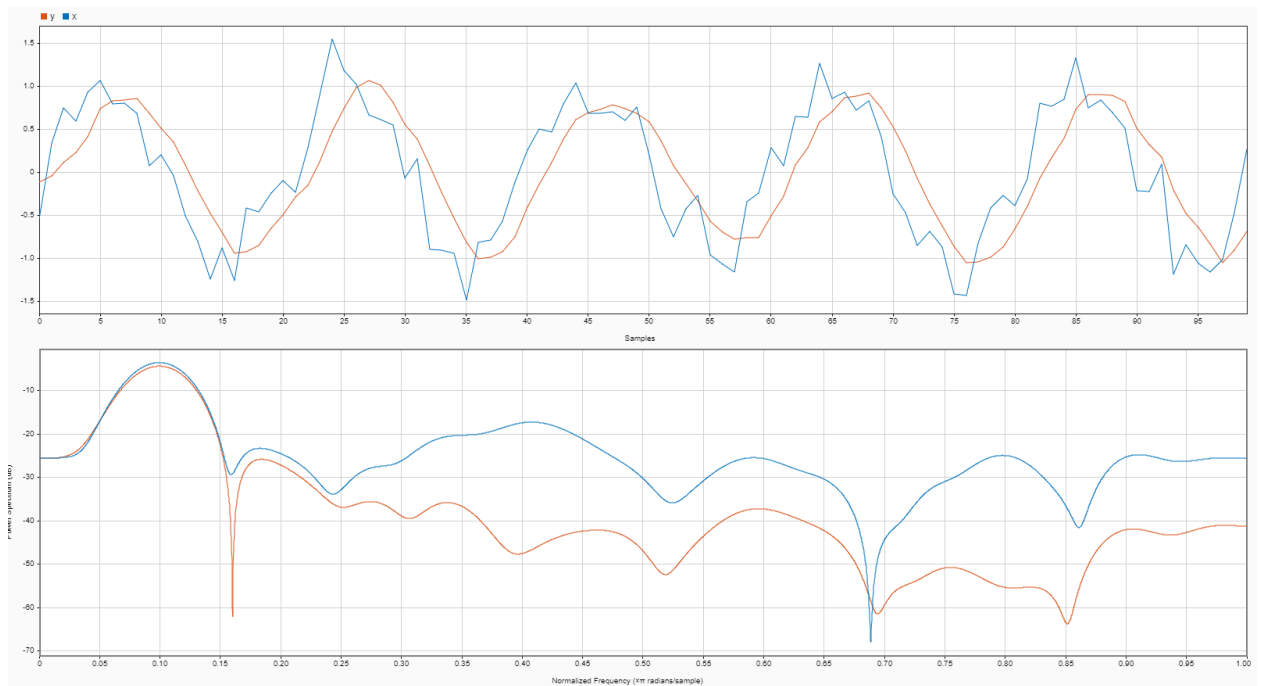
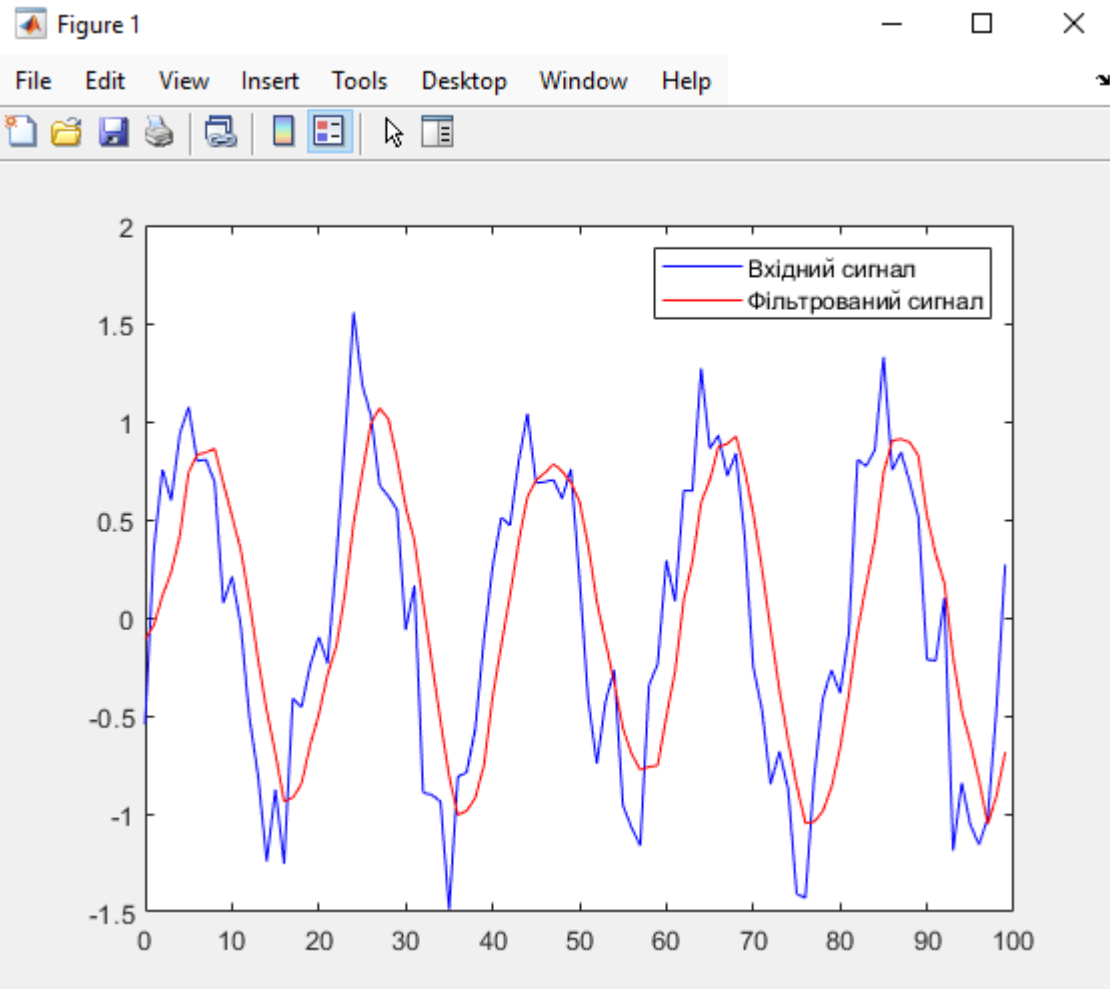
```
% Вхідний сигнал (синусоїдальний сигнал з шумом)
n = 0:99;
x = sin(0.1*pi*n) + 0.25*randn(1,100);

% KIX-фільтр
b = [0.2 0.2 0.2 0.2 0.2]; % Коефіцієнти чисельника
a = 1; % Коефіцієнт знаменника

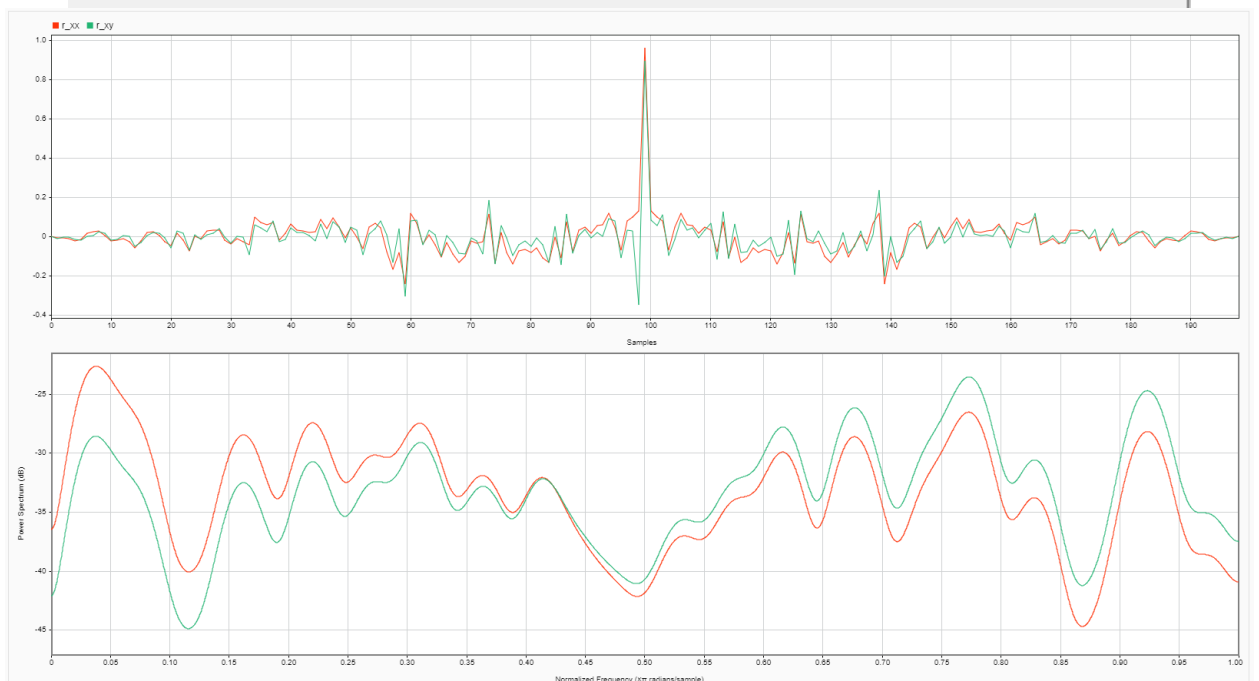
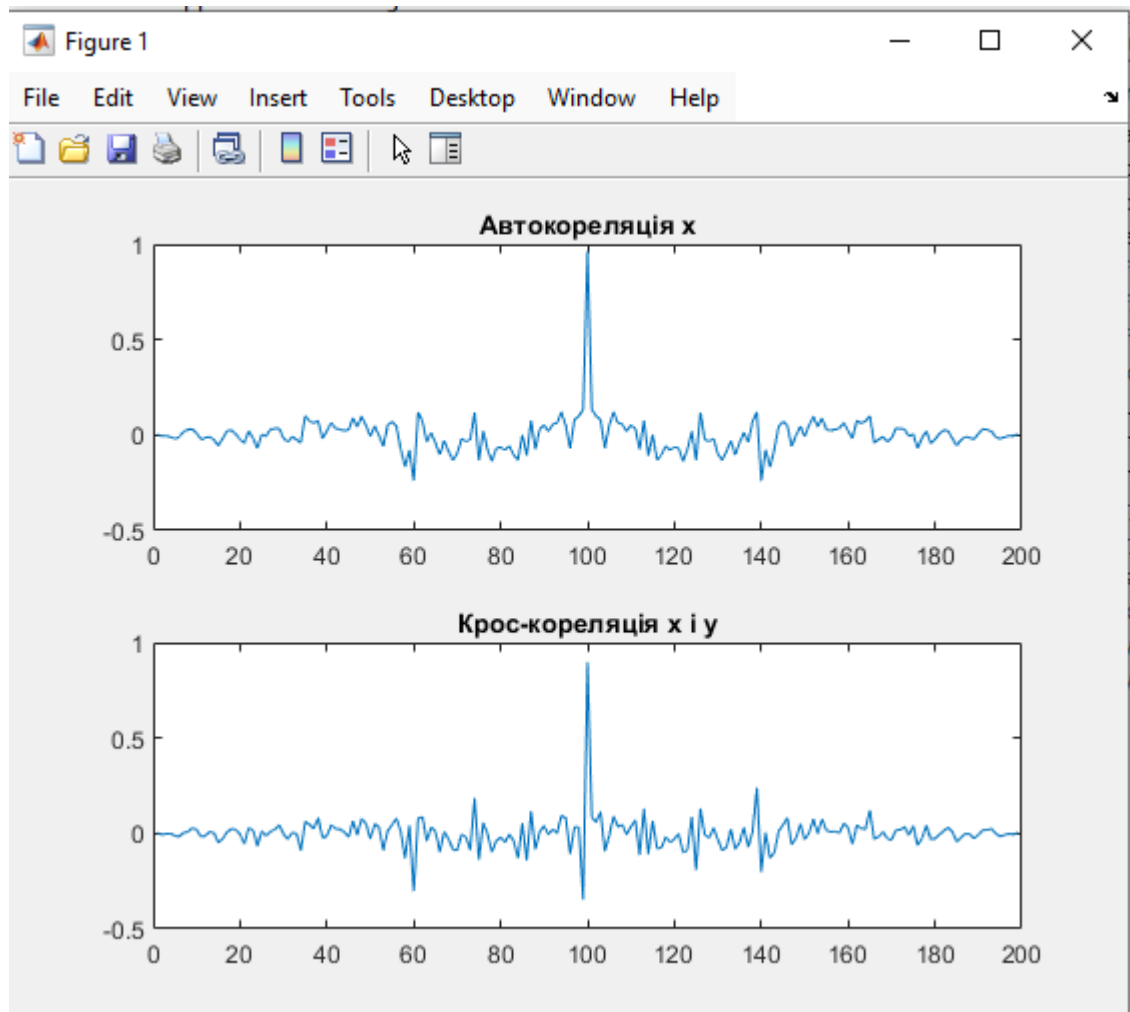
% Фільтрація сигналу
y = filter(b, a, x);

% Графіки
figure;
plot(n, x, 'b', n, y, 'r');
legend('Вхідний сигнал', 'Фільтрований сигнал');
```

Скріни роботи фільтру:



Функція *xcorr* обчислює автокореляцію або крос-кореляцію сигналів. Кореляція використовується для вимірювання подібності між сигналами або між сигналом та його затриманою копією.



Модель фільтра Вінера в MATLAB:

Уявіть, що у нас є ситуація, де ми маємо корисний сигнал, який складається з суми двох дискретних гармонік. Цей сигнал, дуже подібний до суми звуків музики чи різних частот, може бути представлений математично як

$$s = \sin(2\pi \cdot 0.01 \cdot k) + \sin(2\pi \cdot 0.005 \cdot k) \text{ де } k - \text{індекс часу.}$$

Тепер, коли ми спробуємо записати цей сигнал, додаванням до нього шуму, це може схоже на спробу записати музику в умовах навколишнього шуму, який заважає. Щоб симулювати цей процес у MATLAB, ми використовуємо фільтр нижньої частоти (KIX-фільтр), який відфільтровує білий шум, перетворюючи його на корельований шум, що нагадує той, який може з'явитися в записі музики через наявність навколишнього шуму. Також ми генеруємо шум на опорному вході, який виходить з використанням того ж білого шуму, але з допомогою фільтра нижньої частоти третього порядку. Щоб очистити корисний сигнал від цього шуму, ми використовуємо алгоритм Вінера, який дозволяє нам визначити оптимальний фільтр, який видаляє шум, залишаючи лише корисний сигнал. Після виконання алгоритму ми можемо порівняти наш очищений сигнал з оригінальним, а також з сигналом, який був спотворений шумом. Таке порівняння дає нам змогу оцінити ефективність нашого фільтру та якість відновленого сигналу.

Код програми:

```
k = (1:300)';

% Формування корисного сигналу
s = sin(2*pi*0.01*k) + sin(2*pi*0.005*k);

% Отримання некорельованого шуму
n1 = 0.8*randn(300, 1);

% Фільтр для шуму
f1 = [1, 0.5];

% Корельований шум на основному вході
n = filter(1, f1, n1);

% Сума сигналу та шуму
d = s + n;

% Фільтр для шуму на основному вході
f2 = [1, -0.8, 0.4, -0.2];
```

```

% Шум на допоміжному вході
x = filter(f2, 1, n1);

% Порядок фільтра
L = 2;

% Функція firwiener (в MATLAB немає вбудованої функції firwiener, її потрібно
реалізувати самостійно)
fw = firwiener(L-1, x, d); % синтез (обчислення) фільтра Вінера

% Вінерівська фільтрація шуму
yw = filter(fw, 1, x);

% Вінерівська оцінка корисного сигналу
ew = d - yw;

% Побудова графіків
figure;
plot(k, [ew, s, d]);
legend('Фільтр Вінера', 'Корисний сигнал', 'Сигнал із шумом');
title('L = 2');
xlabel('Індекс часу k');
ylabel('Амплітуда');

% Реалізація функції firwiener
function B = firwiener(N, X, Y)
    % Автокореляція вхідного сигналу
    Rxx = xcorr(X, 'biased');
    Rxx = Rxx(length(X):(length(X)+N));

    % Крос-кореляція між вхідним та вихідним сигналами
    Rxy = xcorr(X, Y, 'biased');
    Rxy = Rxy(length(X):(length(X)+N));

    % Розв'язання рівнянь Юлія-Вокера
    Rxx_matrix = toeplitz(Rxx(1:N+1));
    B = Rxx_matrix \ Rxy(1:N+1);
end

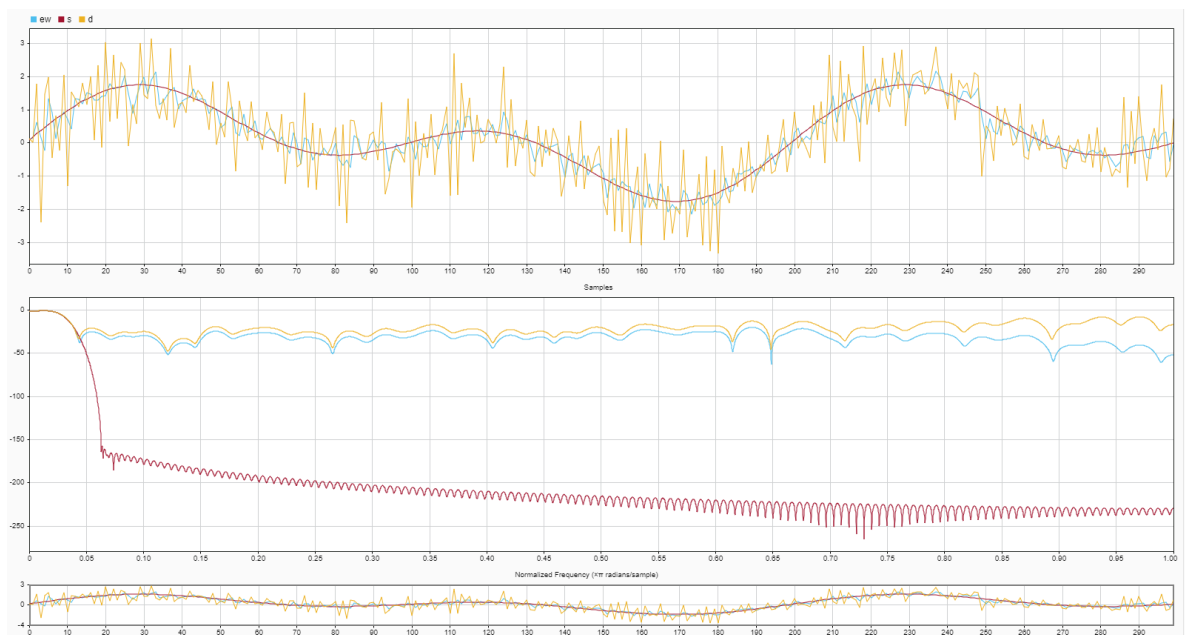
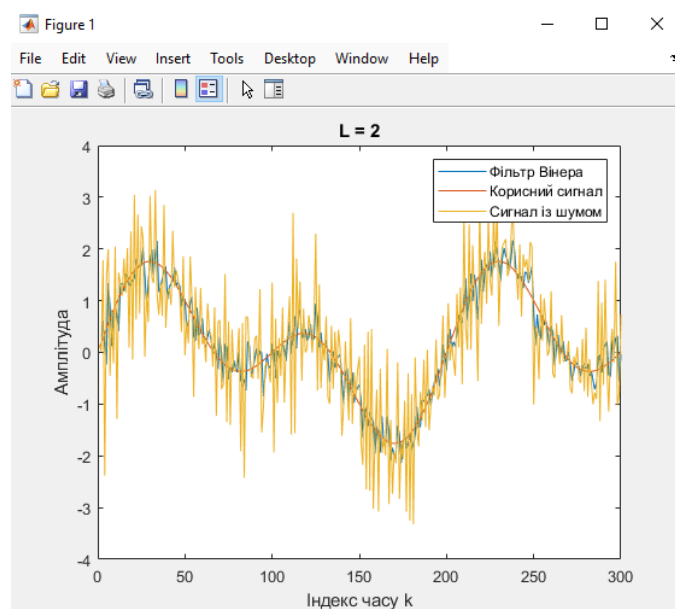
```

Розберемо детально даний код:

1. Спочатку створюється вектор часу k для дискретизації сигналу.
2. Формується корисний сигнал s , що складається з двох дискретних гармонік.
3. Генерується некорельований білий шум $n1$.
4. Шум $n1$ фільтрується фільтром $f1$, що перетворює його на корельований шум n .
5. Сума сигналу s та шуму n формує сигнал зі шумом d .
6. Шум $n1$ фільтрується фільтром $f2$, щоб отримати шум на опорному вході x .
7. Обчислюється фільтр Вінера fw за допомогою функції `firwiener`.

8. Шум на опорному вході x фільтрується за допомогою фільтра Вінера fw , результат зберігається в uw .
9. Вінерівська оцінка корисного сигналу обчислюється як різниця між вхідним сигналом d та фільтрованим шумом uw .
10. Побудова графіка, де порівнюються Вінерівська оцінка, корисний сигнал та сигнал із шумом.
11. Функція `firwiener` реалізує алгоритм розрахунку фільтра Вінера на основі автокореляції та крос-кореляції вхідних сигналів.

Результати роботи(графіки та нормалізована частота) :



На рисунку представлено графік, який показує роботу фільтра Вінера для видалення шуму з сигналу.

Сигнал складається з двох синусоїдальних компонентів з частотами 0,01 Гц і 0,005 Гц.

Шум додається до сигналу в два етапи:

- Некорельований шум: цей шум має нульову кореляцію з сигналом.
- Корельований шум: цей шум має ненульову кореляцію з сигналом.

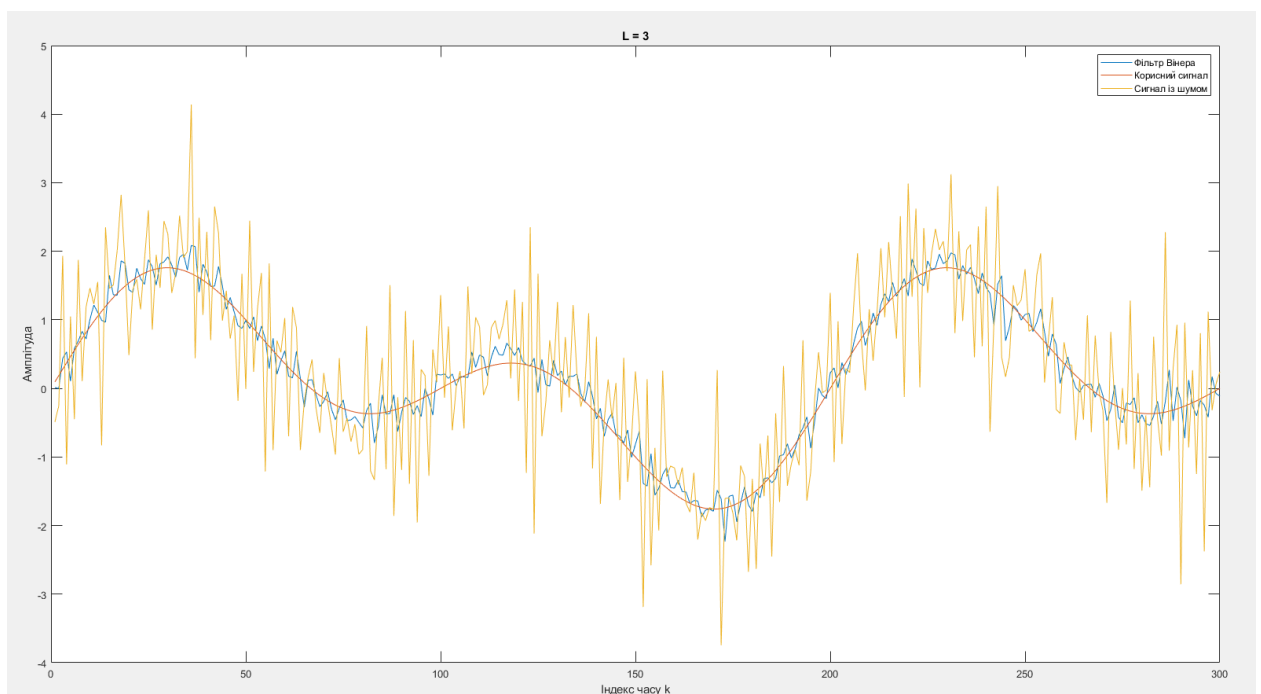
Фільтр Вінера використовується для видалення шуму з сигналу. Фільтр розраховується на основі автокореляції вхідного сигналу та крос-кореляції між вхідним та вихідним сигналами.

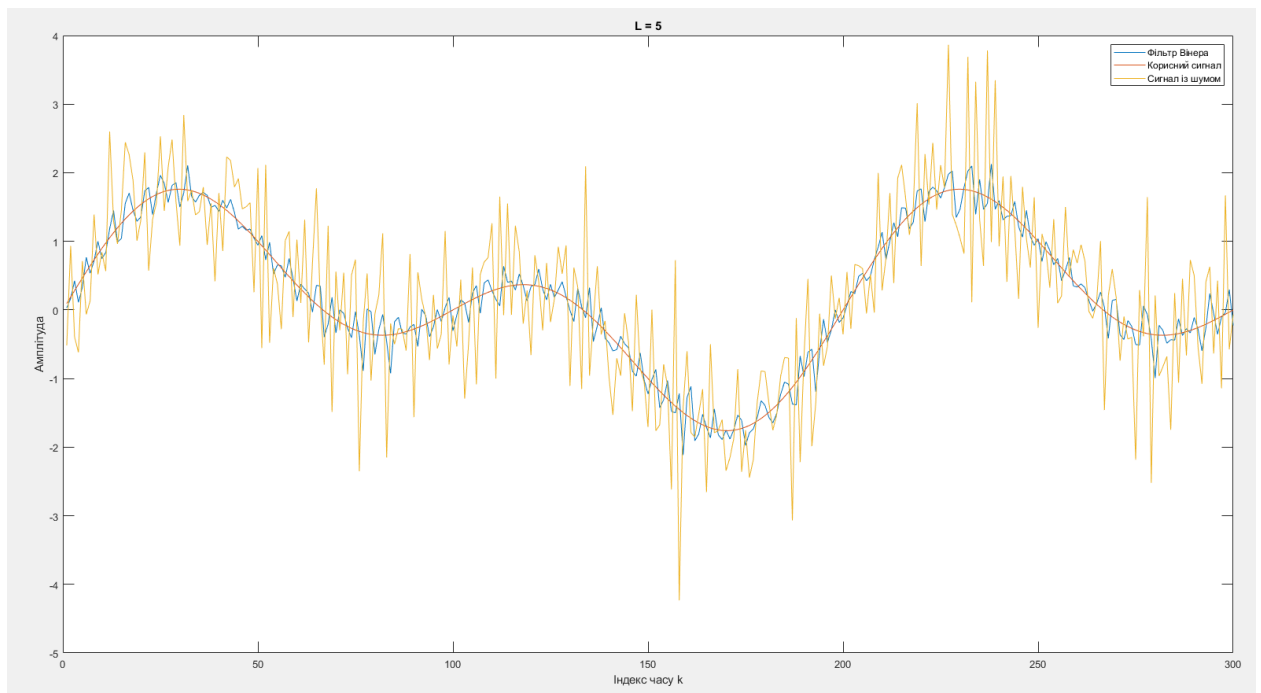
Результати

Фільтр Вінера здатний ефективно видаляти шум з сигналу. Оцінка сигналу, отримана за допомогою фільтра Вінера, значно ближча до початкового сигналу, ніж вихідний сигнал, забруднений шумом.

Проведемо також моделювання ще для двох інших значень порядку фільтра:

$$L = 3$$





При збільшенні значення L у фільтрі Вінера можуть відбуватися такі зміни:

1. **Збільшення різниці між корисним сигналом та шумом:** При збільшенні порядку фільтра L , фільтр Вінера може краще враховувати взаємодію між корисним сигналом та шумом, оскільки він може адаптуватися до більш складних шаблонів вхідного сигналу та шуму.
2. **Більша кількість коефіцієнтів фільтра:** Збільшення порядку фільтра призведе до більшої кількості коефіцієнтів у фільтрі Вінера, що дозволить краще враховувати складніші характеристики вхідного сигналу та шуму.
3. **Збільшення обчислювальної складності:** Збільшення порядку фільтра призведе до збільшення обчислювальної складності обчислення коефіцієнтів фільтра Вінера та застосування фільтрації.
4. **Можлива перенавчаність:** Збільшення порядку фільтра може призвести до перенавчання фільтра на шум, зокрема при дуже великих значеннях L , які можуть недооцінювати значення корисного сигналу.
5. **Покращення якості відновлення сигналу:** У багатьох випадках збільшення порядку фільтра може призвести до покращення якості відновлення корисного сигналу, оскільки фільтр Вінера може краще адаптуватися до складних характеристик сигналу та шуму.

Наведемо приклад моделювання фільтра Вінера для корисного сигналу в вигляді прямокутного імпульсного сигналу:

Код програми:

```
k=(1:500)'; % Створення послідовності від 1 до 500

% формування корисного сигналу (у вигляді прямокутного сигналу)
s2 = square(.06*k, 50);

% Отримання некорельованого шуму
n1 = 0.6 * randn(500, 1);

f1 = [1, 0.7]; % Фільтр для шуму

% Корельований шум на основному вході
n = filter(1, f1, n1);

% Сума сигналу та шуму
d = s2 + n;

% Фільтр для шуму на основному вході
f2 = [1, -0.8, 0.4, -0.2];

% Сигнал на опорному вході
x = filter(f2, 1, n1);

L = 8; % Порядок фільтра

% Розрахунок фільтра Вінера
fw = firwiener(L-1, x, d);

% Вінерівська фільтрація шуму
yw = filter(fw, 1, x);

% Вінерівська оцінка корисного сигналу
ew = d - yw;

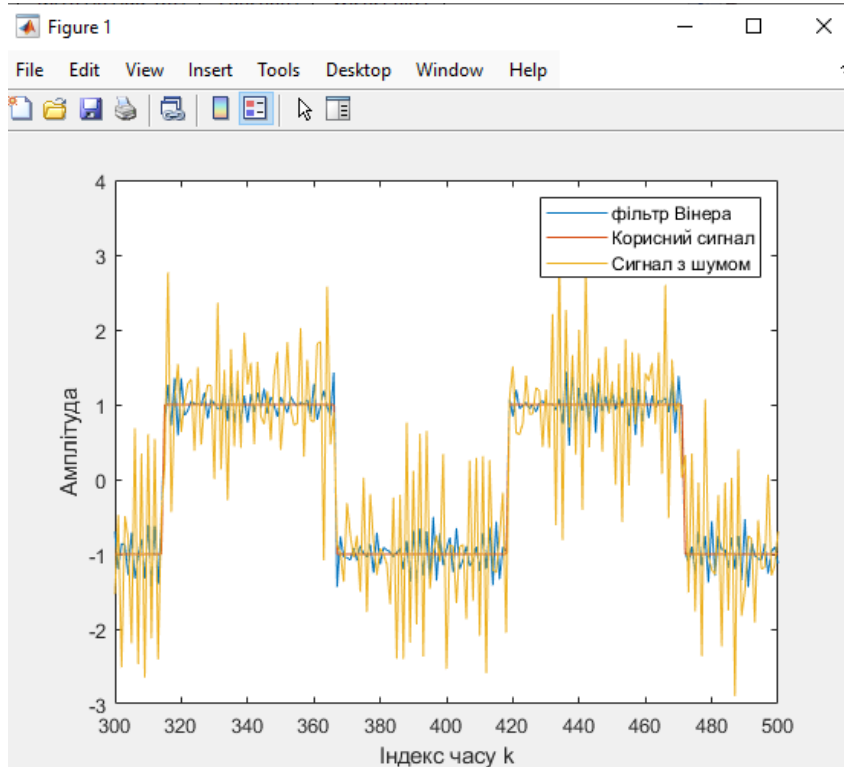
% Побудова графіків
figure
plot(k(300:end), [ew(300:end), s2(300:end), d(300:end)]);
legend('фільтр Вінера', 'Корисний сигнал', 'Сигнал з шумом');
xlabel('Індекс часу k'), ylabel('Амплітуда')

% Реалізація функції firwiener
function B = firwiener(N, X, Y)
    % Автокореляція вхідного сигналу
    Rxx = xcorr(X, 'biased');
    Rxx = Rxx(length(X):(length(X)+N));

    % Крос-кореляція між вхідним та вихідним сигналами
    Rxy = xcorr(X, Y, 'biased');
    Rxy = Rxy(length(X):(length(X)+N));

    % Розв'язання рівнянь Юлія-Вокера
    Rxx_matrix = toeplitz(Rxx(1:N+1));
    B = Rxx_matrix \ Rxy(1:N+1);
end
```

Результати роботи програми:



Деконволюція Вінера для розмитих зображень:

Для цього нам потрібно встановити розширення Image Processing Toolbox. Image Processing Toolbox в MATLAB - це потужний інструментарій для обробки зображень, який надає широкий спектр функцій для аналізу, обробки та візуалізації зображень. Ось деякі з найважливіших функцій, доступних у Image Processing Toolbox:

1. **Читання та запис зображень:** Функції `imread` та `imwrite` використовуються для читання зображень з файлу та запису їх у файл, відповідно. Завдяки підтримці різних форматів зображень, таких як JPEG, PNG, BMP, TIFF тощо, ці функції дозволяють легко взаємодіяти з зображеннями.
2. **Попередня обробка:** Image Processing Toolbox містить низку функцій для попередньої обробки зображень, таких як згладжування, видалення шуму, зміна розміру, видалення фону тощо. Функції, які представляють цю функціональність, включають `imfilter`, `imnoise`, `imresize`, `imadjust` та інші.
3. **Обробка та аналіз кольору:** Для роботи з кольоровими зображеннями доступні функції для перетворення кольорових просторів, витягування каналів кольору, а також аналіз кольору. Функції `rgb2gray`, `rgb2hsv`, `imhist` та інші допомагають в аналізі кольорових характеристик зображень.
4. **Сегментація зображень:** Image Processing Toolbox містить інструменти для сегментації зображень, яка дозволяє виділити об'єкти на зображенні. Функції, такі як `imbinarize`, `imsegthresh`, `watershed`, допомагають в цьому процесі.
5. **Операції над областями зображень:** Для виконання різноманітних операцій над областями зображень доступні функції, такі як морфологічні операції (розширення, звуження, відкриття, закриття), алгоритми знаходження контурів та регіонів, а також операції над еліпсами, прямокутниками тощо.

6. **Фільтрація та відновлення зображень:** Функції для фільтрації зображень, такі як фільтри Гауса, медіанні фільтри, фільтри собеля, а також методи відновлення зображень, такі як деконволюція та усунення розмиття.
7. **Морфологічна обробка:** Image Processing Toolbox надає функції для виконання морфологічних операцій, таких як видалення шуму, виділення об'єктів, заповнення отворів, а також видалення вузьких мостів і малих об'єктів.
8. **Функції для візуалізації зображень:** У Image Processing Toolbox є інструменти для візуалізації зображень та їх характеристик, такі як `imshow`, `imtool`, `improfile` тощо, які дозволяють відображати та аналізувати зображення у зручний спосіб.

Цей інструментарій надає великі можливості для обробки та аналізу зображень у MATLAB.

Код програми:

```
% Читання первинного зображення
Ioriginal = imread('cameraman.tif');

% Імітація розмиття від руху без шуму
len = 21;
theta = 11;
PSF = fspecial('motion', len, theta);
Idouble = im2double(Ioriginal);
blurred = imfilter(Idouble, PSF, 'conv', 'circular');

% Відновлення розмитого зображення без шуму
wnr1 = deconvwnr(blurred, PSF);

% Імітація розмиття від руху і гаусівського шуму
noise_mean = 0;
noise_var = 0.0001;
blurred_noisy = imnoise(blurred, 'gaussian', noise_mean, noise_var);

% Відновлення розмитого зашумленого зображення без оцінки шуму
wnr2 = deconvwnr(blurred_noisy, PSF);

% Відновлення розмитого зашумленого зображення з оцінкою шуму
signal_var = var(Idouble(:));
NSR = noise_var / signal_var;
wnr3 = deconvwnr(blurred_noisy, PSF, NSR);

% Імітація розмиття від руху і квантуючого шуму 8-бітного зображення
blurred_quantized = imfilter(Ioriginal, PSF, 'conv', 'circular');

% Відновлення розмитого квантованого зображення без оцінки шуму
wnr4 = deconvwnr(blurred_quantized, PSF);
```

```

% Відновлення розмитого квантованого зображення з оцінкою шуму
uniform_quantization_var = (1/256)^2 / 12;
NSR = uniform_quantization_var / signal_var;
wnr5 = deconvwnr(blurred_quantized, PSF, NSR);

% Виведення усіх зображень на одному полотні
figure;

subplot(3, 3, 1);
imshow(Ioriginal);
title('Оригінальне зображення');

subplot(3, 3, 2);
imshow(blurred);
title('Розмите зображення');

subplot(3, 3, 3);
imshow(wnr1);
title('Відновлене розмите зображення без шуму');

subplot(3, 3, 4);
imshow(blurred_noisy);
title('Розмите і зашумлене зображення');

subplot(3, 3, 5);
imshow(wnr2);
title('Відновлення розмитого зашумленого зображення (NSR = 0)');

subplot(3, 3, 6);
imshow(wnr3);
title('Відновлення розмитого зашумленого зображення (оцінений NSR)');

subplot(3, 3, 7);
imshow(blurred_quantized);
title('Розмите квантоване зображення');

subplot(3, 3, 8);
imshow(wnr4);
title('Відновлення розмитого квантованого зображення (NSR = 0)');

subplot(3, 3, 9);
imshow(wnr5);
title('Відновлення розмитого квантованого зображення (оцінений NSR)');

```

У даному коді використовуються наступні функції та їх функціонал:

1. ***fspecial('motion', len, theta)***: Ця функція створює розмітку зображення, що виникає від руху, з заданими параметрами. *len* визначає довжину розмиття, а *theta* - кут руху.
2. ***im2double(Ioriginal)***: Ця функція перетворює вхідне зображення в подвійне (типу *double*), щоб забезпечити правильну роботу з обробкою та відновленням зображення.

3. ***imfilter(Idouble, PSF, 'conv', 'circular')***: Ця функція застосовує фільтр до зображення. В даному випадку використовується операція згортки ('conv'), а розмиття зображення відбувається в циркулярному режимі ('circular'), щоб уникнути артефактів на краях зображення.
4. ***imnoise(blurred, 'gaussian', noise_mean, noise_var)***: Ця функція додає гаусівський шум до розмитого зображення з заданим середнім значенням (noise_mean) та дисперсією шуму (noise_var).
5. ***deconvwnr(blurred, PSF)***: Ця функція виконує відновлення розмитого зображення без оцінки співвідношення сигнал-шум. Вона використовує алгоритм рекурсивної інверсії Вінера для відновлення.
6. ***deconvwnr(blurred_noisy, PSF, NSR)***: Ця функція також відновлює розмите зашумлене зображення, але враховує оцінку співвідношення сигнал-шум (NSR). Воно користується алгоритмом рекурсивної інверсії Вінера, але з використанням оцінки NSR для поліпшення якості відновлення.

Ці функції дозволяють вам виконувати операції з обробки та відновлення зображень з використанням Image Processing Toolbox у MATLAB.

Проаналізуємо даний код:

У даному коді реалізовано приклад застосування фільтра Вінера для відновлення розмитих зображень. Цей приклад показує різні сценарії, включаючи випадки з різними рівнями шуму та квантуванням.

Фільтр Вінера - це метод деконволюції, який використовується для відновлення зображень, що були розмиті або зіпсовані шумом. Основна ідея полягає в тому, щоб врахувати спектральні характеристики шуму та сигналу для відновлення оригінального зображення.

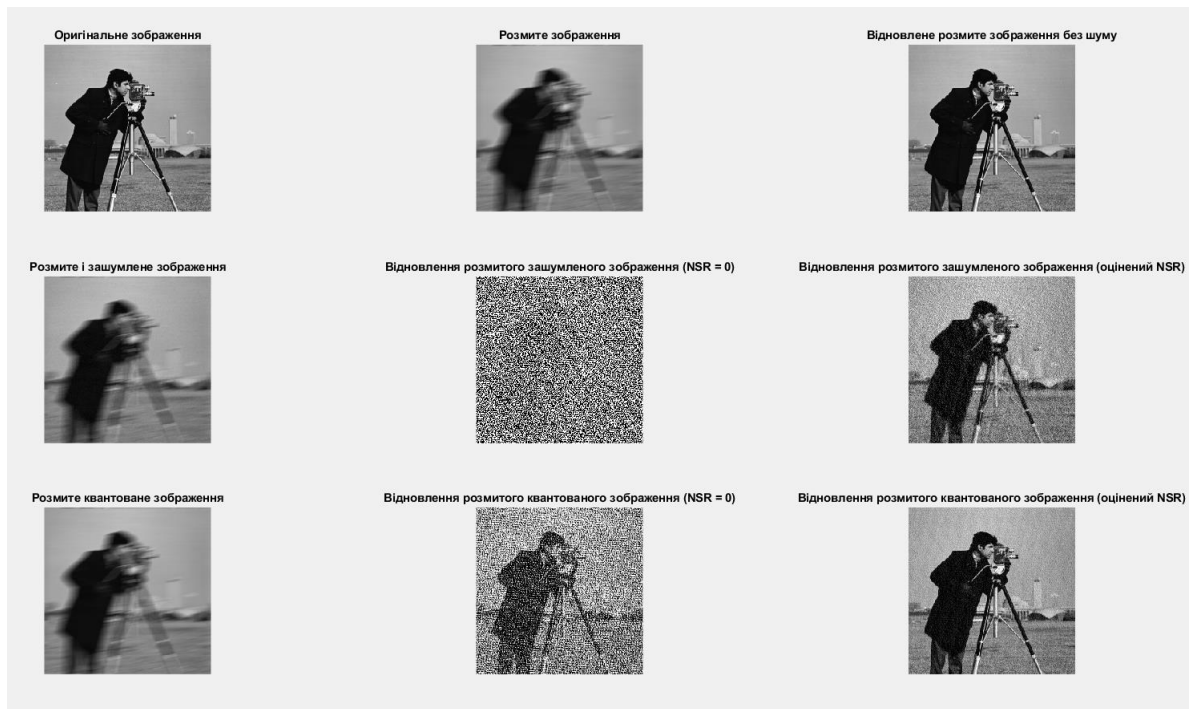
У першому випадку, коли розмите зображення не має шуму, фільтр Вінера успішно відновлює зображення, використовуючи лише точно відому функцію розсіювання точки (PSF).

У другому випадку, коли до розмитого зображення додається гаусівський шум, без врахування оцінки шуму фільтр Вінера погіршує якість відновленого зображення через посилення шуму. Однак, при належній оцінці співвідношення сигнал-шум, фільтр Вінера може ефективно відновити зображення, компенсуючи ефект шуму.

У третьому випадку, коли розмите зображення використовується в оригінальному форматі `uint8` з квантуванням, навіть без додавання шуму, фільтр Вінера може погіршити якість відновлення через вплив квантування. Знову ж таки, правильна оцінка співвідношення сигнал-шум допомагає зменшити цей вплив і покращує якість відновленого зображення.

Отже, фільтр Вінера є потужним інструментом для відновлення розмитих зображень, але ефективність його застосування значно залежить від належної оцінки характеристик шуму та сигналу в зображенні. Деконволюція Вінера може бути ефективною, коли частотні характеристики зображення та адитивного шуму відомі, принаймні до певного ступеня.

Результат роботи програми:



Висновки

В результаті роботи над проектуванням оптимальних цифрових фільтрів Вінера в системі MATLAB, було досліджено основні поняття та застосування цифрових фільтрів у сучасному світі. Цифрові фільтри відіграють важливу роль у системах обробки сигналів, дозволяючи видобувати корисну інформацію з шуму, покращувати якість переданих даних та створювати нові сигнали з заданими характеристиками. Основними цілями фільтрації є покращення якості сигналу, вилучення інформації, розділення сигналів та формування сигналу. Фільтри можуть бути різних типів, таких як низькочастотні, високочастотні, смугові, загороджувальні та адаптивні. Фільтр Вінера є оптимальним для зменшення шуму в сигналах при наявності стаціонарного шуму та корисного сигналу. Він характеризується припущенням, що сигнал та шум є стаціонарними лінійними випадковими процесами з відомими спектральними характеристиками, а критерієм ефективності є мінімальна середньоквадратична помилка. Алгоритм фільтра Вінера полягає в визначенні коефіцієнтів фільтра, які максимально зменшують похибку між вихідним сигналом та зразковим сигналом. Це здійснюється шляхом статистичного усереднення квадрата похибки. Практичне використання фільтра Вінера обмежене через необхідність визначення автокореляційної матриці та взаємної кореляції, а також транспонування матриць, що є трудомістким процесом. Розробка цифрового фільтра відбувається у п'ять етапів: специфікація вимог до фільтру, обчислення відповідних коефіцієнтів фільтра, подання фільтра відповідною структурою, аналіз впливу кінцевої розрядності на продуктивність фільтра та реалізація фільтра на програмному та/або апаратному рівні. Також фільтр Вінера є важливим інструментом у багатьох галузях, таких як стиснення даних, біомедична обробка сигналів, обробка мови та зображень.

Список використаних джерел

1. Державний університет "Житомирська політехніка" - Освітній портал.
URL: https://learn.ztu.edu.ua/pluginfile.php/242980/mod_folder/content/0/Лекція%202%20СЦОС%203%20БІ.pdf?forcedownload=1 (дата звернення: 23.05.2024).
2. ELARTU – Інституційний репозитарій ТНТУ імені Івана Пулюя: Домівка.
URL: <https://elartu.tntu.edu.ua/bitstream/lib/22528/1/lab07.pdf> (дата звернення: 27.05.2024).
3. DSpace :: ELAKPI :: Репозитарій КПІ ім. Ігоря Сікорського.
URL: <https://ela.kpi.ua/server/api/core/bitstreams/ff6d2e31-1e7b-4b1b-8ea3-51aabbe3c7b9/content> (дата звернення: 27.05.2024).
4. Deblur Images Using a Wiener Filter- MATLAB & Simulink Example. MathWorks - Makers of MATLAB and Simulink - MATLAB & Simulink.
URL: <https://www.mathworks.com/help/images/deblurring-images-using-a-wiener-filter.html> (дата звернення: 27.05.2024).
5. Using Signal Analyzer App- MATLAB & Simulink. MathWorks - Makers of MATLAB and Simulink - MATLAB & Simulink. URL: <https://www.mathworks.com/help/signal/ug/using-signal-analyzer-app.html> (дата звернення: 29.05.2024).
6. PTC Help Center. Access Denied.
URL: https://support.ptc.com/help/mathcad/r8.0/en/index.html#page/PTC_Mathcad_Help/wiener_filtering.html (дата звернення: 30.05.2024).
7. Ряд Вінера. www.wikidata.uk-ua.nina.az. URL: https://www.wikidata.uk-ua.nina.az/Ряд_Вінера.html (дата звернення: 31.05.2024).
8. Наукові праці Вінницького національного технічного університету.
URL: <https://praci.vntu.edu.ua/index.php/praci/article/download/509/504/> (дата звернення: 31.05.2024).